MODEL RESEARCH AND SELECTION FOR AGENTIC AI

AI-Powered Workforce Analytics & Talent Intelligence Dashboard

Internship Research Document

Purpose: Research and recommend the models, frameworks, tools, and architecture required to add an Agentic AI layer to an already-developed workforce analytics visualization/dashboard.

Prepared for internship project documentation

1. Executive Summary

The Workforce Analytics & Talent Intelligence Dashboard provides visual insights into employee and organizational data. The next stage is to introduce an Agentic AI layer that allows users to interact with analytics using natural language and receive data-driven answers, explanations, predictions, and recommendations.

The recommended approach is not to train a large language model from scratch. Instead, an existing foundation model should be used as the reasoning and orchestration engine. The agent can call specialized tools for SQL/data analysis, Retrieval-Augmented Generation (RAG), machine-learning predictions, and other approved functions.

The architecture separates responsibilities: the LLM handles language understanding, reasoning, planning, tool selection, and response generation; RAG provides knowledge from documents; ML models provide specialized predictions; SQL/Python tools provide factual calculations from structured workforce data; and the existing dashboard remains the visualization layer.

2. Project Context

The project is an AI-Powered Workforce Analytics & Talent Intelligence Dashboard. The visualization component is assumed to be substantially completed. The Agentic AI component will sit above the existing analytics and provide a natural-language interface to approved workforce information.

Primary objectives of the AI layer:

- Answer workforce questions in natural language.
- Retrieve information from HR policies, documents, and approved knowledge sources.
- Perform calculations and analytical queries against structured workforce data.
- Use predictive ML models such as attrition prediction when appropriate.
- Generate explanations and actionable recommendations.
- Select the correct tool instead of answering from unsupported assumptions.
- Maintain traceability by identifying the data or tool used for important analytical answers.

3. What Is Agentic AI?

Agentic AI refers to an AI system that can interpret a user's goal, decide what actions are needed, use available tools, observe their results, and produce a final response. Unlike a simple chatbot, an agent can follow a multi-step workflow and select different capabilities depending on the question.

Example: If a user asks, “Which department has the highest attrition and why?”, the agent may first query workforce data, calculate department-level attrition, identify the highest-risk department, retrieve relevant HR context, and then generate an explanation. The workflow should be controlled by application rules and tool permissions.

4. Why an LLM Is Required

The LLM is the central language and reasoning component. It converts natural-language requests into structured intentions, decides which available tool is relevant, interprets tool results, and communicates the final result in understandable language.

Typical responsibilities:

- Intent understanding and query classification.
- Reasoning over multi-step tasks.
- Tool/function selection.
- Natural-language generation.

- Summarization and explanation.
- Combining retrieved knowledge with analytical results.
- Following system instructions, business rules, and output formats.

The LLM should not be treated as the source of truth for numerical workforce metrics. Numbers should preferably come from the database, analytics layer, or a validated ML model.

5. Foundation / LLM Model Research

Candidate families for research include OpenAI GPT models, Google Gemini, Anthropic Claude, Meta Llama, and Mistral models. The final selection should be based on deployment requirements, available APIs, budget, data privacy requirements, latency, context length, structured-output support, and tool/function-calling capabilities.

Evaluation criterion	Why it matters
Reasoning quality	Important for multi-step agent workflows and interpreting analytical results.
Tool/function calling	Allows the agent to invoke SQL, RAG, Python, prediction, and other approved tools.
Structured outputs	Useful when the application needs predictable JSON or structured tool arguments.
Context window	Important for long documents, retrieved context, and complex conversations.
Latency	Affects user experience in an interactive dashboard.
Cost	Important for repeated enterprise queries and internship deployment constraints.
Privacy/security	Workforce data may be confidential; data handling and provider policies must be reviewed.
Deployment options	Hosted APIs and self-hosted/open-weight models have different operational trade-offs.

Recommendation: For an internship prototype, prioritize a strong API-based instruction-following LLM with reliable tool/function calling rather than training a foundation model. The exact provider/model should be selected after testing representative project questions.

6. Embedding Model Research

Embeddings convert text into numerical vectors that capture semantic meaning. They are primarily required for the RAG component, not for the core reasoning of the agent.

Example flow: HR policy documents → chunking → embedding model → vector database → similarity search → relevant chunks → LLM.

Selection criteria: semantic retrieval quality, multilingual support if required, embedding dimension, inference cost, latency, licensing, and compatibility with the selected vector database.

Candidate families include sentence-transformer models, hosted embedding APIs, and other enterprise embedding models. The model should be evaluated using real HR/workforce questions and expected source documents.

7. Retrieval-Augmented Generation (RAG)

RAG should be used when the agent needs information stored in documents rather than directly in structured workforce tables. Examples include HR policies, benefits documents, onboarding material, organizational guidelines, and approved internal documentation.

RAG components:

- Document loader: reads approved documents.
- Chunking: splits documents into searchable sections.
- Embedding model: converts chunks into vectors.

- Vector database: stores embeddings and metadata.
- Retriever: finds relevant chunks for a user question.
- Prompt/context construction: supplies retrieved content to the LLM.
- Generation: LLM answers using the retrieved context.

RAG reduces the need to put all company knowledge directly into the model and can improve grounded responses when retrieval and prompting are implemented correctly.

8. Machine Learning Models for Workforce Analytics

The agent may need specialized ML models for predictive analytics. These models should be treated as tools that the agent can call when a question requires a prediction or scoring operation.

Use case	Models to research	Agent output
Attrition prediction	Logistic Regression, Random Forest, XGBoost	Risk probability / risk category
Performance prediction	Random Forest, Gradient Boosting, XGBoost	Predicted performance or score
Employee segmentation	K-Means, hierarchical clustering	Employee groups/segments
Anomaly detection	Isolation Forest, statistical methods	Potential unusual patterns
Skill-gap analysis	Embeddings + similarity / classification	Missing or related skills
Resume/job matching	Sentence Transformers / embedding models	Similarity or match score
Sentiment analysis	BERT-family or other NLP classifiers	Sentiment category/score

Model selection should be based on the dataset, target variable, class imbalance, interpretability, accuracy, calibration, and business requirements. For high-impact employment decisions, predictions should support human review rather than automatically making employment decisions.

9. SQL and Data-Analysis Tools

Structured workforce information should generally be queried through a controlled data-access layer. The agent can translate natural-language questions into structured queries or invoke predefined analytical functions.

Examples: “What is the average salary by department?”, “How many employees joined this year?”, and “Which location has the highest attrition?” are best answered from the authoritative structured data source rather than from the LLM's memory.

A safer implementation uses validated query templates, schema-aware generation, read-only database credentials, query limits, access controls, and result validation.

10. Agent Framework Research

A framework can provide the orchestration layer between the LLM and tools.

Framework / approach	Potential use	Research focus
LangGraph	Stateful, controlled agent workflows	Graph/state management, tool calls, human approval, retries
LangChain	LLM applications and tool integrations	Prompts, retrievers, tools, model integration
CrewAI	Multi-agent role-based workflows	Agent collaboration and task delegation
AutoGen	Conversational/multi-agent systems	Agent coordination and tool-based workflows
Custom Python orchestration	Small controlled prototype	Maximum control, more development effort

Recommended research direction: LangGraph + an LLM is a strong candidate when the project requires controlled, multi-step agent workflows. LangChain components can be used alongside it for model, retriever, and tool integrations.

11. Proposed Agent Architecture

High-level architecture:

User → Dashboard/Chat Interface → Agent/LLM → Intent & Planning → Tool Selection → SQL / RAG / ML Prediction / Python Analytics → Tool Result Validation → LLM Response → User

Tool routing examples:

- Policy question → RAG retriever.
- Numerical workforce question → SQL/data-analysis tool.
- Attrition-risk question → attrition prediction model.
- Skill similarity question → embedding/similarity tool.
- Trend or calculation question → Python/analytics tool.
- General explanation → LLM without external tool, where appropriate.

The agent should not automatically access every tool. Tool permissions should be explicitly defined, and sensitive actions should require additional controls or human approval.

12. Example Agent Workflow

User: “Which department has the highest employee attrition?”

Step 1: LLM identifies the task as a structured workforce-data query.

Step 2: Agent selects the SQL/analytics tool.

Step 3: Tool calculates attrition by department from the authoritative dataset.

Step 4: Agent receives the result and checks that the response is supported by the returned data.

Step 5: LLM generates a concise natural-language answer and, where possible, explains the metric used.

Second example: “What is the company's leave policy?” → Agent selects RAG → retrieves relevant policy sections → LLM answers using retrieved context.

13. Recommended Model Stack

Layer	Recommended direction	Reason
LLM / Agent brain	Strong tool-calling instruction-following LLM	Reasoning, routing, response generation
Agent orchestration	LangGraph	Controlled multi-step workflows
Embeddings	Evaluate a strong sentence-transformer or hosted embedding model	Semantic document retrieval
Vector store	Chroma / FAISS for prototype; enterprise vector DB as required	RAG retrieval
Structured analytics	SQL + Python/Pandas	Reliable calculations from workforce data
Predictive ML	XGBoost / Random Forest / Logistic Regression depending on task	Specialized workforce predictions
UI	Existing dashboard + chat interface	Natural-language access to analytics

14. Model Selection Methodology

Before final implementation, candidate models should be evaluated using a representative test set created from expected user questions.

- Create 20–50 representative workforce questions.
- Separate questions into SQL, RAG, prediction, analytical, and general categories.
- Measure answer correctness and tool-selection accuracy.
- Measure retrieval relevance for RAG questions.
- Measure ML performance using appropriate validation metrics.
- Record response latency and approximate cost.
- Check robustness against ambiguous or unsupported questions.
- Verify that sensitive data is handled according to organizational requirements.
- Select the smallest/most economical model that meets the required quality.

15. Evaluation Metrics

LLM/Agent: answer correctness, tool-selection accuracy, structured-output validity, latency, cost per request, and failure rate.

RAG: retrieval precision/recall or suitable retrieval metrics, groundedness, answer faithfulness, and source correctness.

Classification: accuracy, precision, recall, F1-score, ROC-AUC where appropriate.

Regression: MAE, RMSE, R^2, and calibration where relevant.

Clustering: silhouette score plus business interpretability.

For workforce predictions, evaluation should also consider fairness, explainability, data leakage, and whether the model is suitable for the intended decision context.

16. Security, Privacy and Responsible AI

Workforce data can contain sensitive employee information. The agent architecture should therefore be designed with privacy and access control from the beginning.

- Use role-based access control.
- Expose only the minimum data needed for each tool.
- Use read-only database access where possible.
- Avoid sending unnecessary personal information to external model APIs.
- Log tool calls and important model decisions for auditing.
- Validate generated SQL before execution.
- Prevent prompt injection from documents and user inputs.
- Do not allow the LLM to make autonomous employment decisions.
- Provide human review for high-impact recommendations.
- Define data retention and secret-management practices.

17. Proposed Implementation Roadmap

- Phase 1 – Finalize data sources and define 20–50 representative user questions.
- Phase 2 – Build controlled SQL/data-analysis tools.

- Phase 3 – Build document ingestion, chunking, embeddings, and vector retrieval.
- Phase 4 – Select and evaluate the LLM.
- Phase 5 – Implement agent orchestration and tool routing.
- Phase 6 – Integrate predictive ML models as callable tools.
- Phase 7 – Connect the agent to the existing dashboard/chat interface.
- Phase 8 – Evaluate accuracy, latency, cost, security, and failure cases.
- Phase 9 – Add monitoring, logging, guardrails, and human approval where required.

18. Final Recommendation

For the internship prototype, the recommended architecture is an existing high-quality tool-calling LLM as the agent's reasoning engine, LangGraph for controlled orchestration, RAG for document-based knowledge, SQL/Python tools for structured workforce analytics, and task-specific ML models for predictions such as attrition risk.

The most important design principle is to keep the LLM responsible for understanding and orchestration while keeping factual calculations and predictions in reliable external tools. This produces a more controllable architecture than expecting the LLM to calculate, remember, and predict everything by itself.

The final model should be selected through a small benchmark using actual project questions rather than by choosing a model solely from general popularity.

19. Conclusion

The Agentic AI layer can transform the completed workforce visualization into an interactive intelligence system. Users can ask questions in natural language while the agent determines whether it should retrieve knowledge, query workforce data, perform an analysis, or invoke a predictive model. This architecture combines LLM reasoning with reliable data and specialized AI models, creating a practical path from dashboard visualization to AI-powered workforce intelligence.

Note: Model names and provider capabilities change over time. Final implementation should verify current API availability, pricing, licensing, context limits, tool-calling support, and organizational data policies before deployment.